C−− (C Minus Minus): A Statically Checked Imperative Language Achieving Memory Safety by Removal

Ronit Mexson
School of Computer Science and Engineering (SCOPE)
Reg. No. 24BAI0036

Mit Kumar Raju Panchal
School of Computer Science and Engineering (SCOPE)
Reg. No. 24BAI0279

Garv Gupta
School of Computer Science and Engineering (SCOPE)
Reg. No. 24BDS0169

***Abstract*—Memory-safety defects remain the dominant source of severe vulnerabilities in software written in C and C++. Existing remedies either add machinery to the language (garbage collection, ownership and borrow checking, fat pointers) or detect faults after the fact (sanitizers). This paper presents C−− (C Minus Minus), a small, statically typed, C-like imperative language built on the opposite principle, *safety by removal*: the constructs that make memory corruption and undefined behaviour expressible are deleted from the grammar instead of being guarded. C−− has no pointers, no null, no uninitialized variables, no implicit conversions and no undefined behaviour; bindings are immutable by default; collections have deep-copy value semantics; and every remaining fault (integer overflow, division by zero, out-of-range indexing, unbounded recursion) is a deterministic, positioned runtime trap. We describe the language, its zero-dependency Python implementation (lexer, recursive-descent parser, semantic analyzer, reference interpreter and ISO C99 transpiler), a diagnostics system with 60 documented error codes written for novices, and a browser-based compiler studio. The evaluation maps eleven common weakness classes to the mechanism that eliminates or traps them, reports a 445-test suite (444 passing, with the remaining failure analysed), and measures the transpiled C99 against both the interpreter and hand-written unchecked C.**

***Index Terms*—programming languages, memory safety, compilers, static analysis, language design, transpilation, compiler diagnostics**

I. Introduction

For decades, systems software has been written in C and C++, languages whose flexibility comes from raw pointers, manual memory management and a large catalogue of undefined behaviour. The cost is well documented: Microsoft has reported that roughly 70% of the vulnerabilities it assigns a CVE each year are memory-safety issues [1], the Chromium project reports a similar share among its serious security bugs [2], and government agencies now urge vendors to publish memory-safety roadmaps [3].

Two families of remedies dominate. *Additive* approaches extend the language with new machinery: safe dialects such as Cyclone [4], CCured [5] and Checked C [6] add annotated pointer kinds and runtime metadata, while Rust [7] adds ownership, borrowing and lifetimes. *Detective* approaches such as AddressSanitizer [8] keep the language unchanged and find faults during testing. Both are effective, but the first raises the conceptual burden on programmers and the second offers no guarantee in production.

This paper explores a third option for the setting of education and small, high-assurance programs: *safety by removal*. Instead of adding mechanisms that make dangerous features safe, C−− removes the dangerous features and keeps only constructs whose behaviour is fully specified. The guiding question is how much of C's explicit, predictable programming style survives when every construct with undefined behaviour is taken away. The contributions are:

- the design of C−−, a statically typed imperative language without pointers, null, uninitialized state, implicit conversions or undefined behaviour, with immutable-by-default bindings and deep-copy value semantics for collections (Section III);
- a complete, dependency-free implementation with two execution back ends: a normative tree-walking interpreter and an ISO C99 [9] transpiler with checked arithmetic (Section IV);
- a diagnostics system of 60 documented error codes that explains each error in plain language and proposes a concrete fix (Section V);
- an evaluation of the safety guarantees, test coverage and performance cost of the approach (Section VI).

The name follows the language's philosophy, *C minus the dangerous parts*. It is unrelated to the portable assembly language of the same name used inside the Glasgow Haskell Compiler [10].

II. Related Work

Safe C dialects. Cyclone [4] restricts pointer arithmetic, adds region-based memory management and several pointer kinds. CCured [5] infers pointer kinds and inserts run-time checks for legacy code, and Checked C [6] adds checked pointer and array types that can be adopted incrementally. These systems retain pointers and therefore must track them; C−− removes them.

Ownership-based languages. Rust [7] achieves memory safety without garbage collection through ownership and bor-

TABLE I
TYPES OF C−− V0.2.0

Type	Semantics
`int`	64-bit signed; overflow traps (R001)
`float`	IEEE 754 binary64; never Inf or NaN (R002, R004)
`bool`	`true`/`false`; never converted to or from numbers
`string`	immutable Unicode text; indexing by code point, `.len()`
`array<T>`	fixed length, bounds-checked indexing (R006)
`list<T>`	growable; `push`, `pop`, `insert`, `remove`, `contains`
`stack<T>`	LIFO; `push`, `pop`, `peek`; empty pop traps (R012)
`queue<T>`	FIFO; `enqueue`, `dequeue`, `peek`

rowing, at the price of lifetime reasoning that is notoriously hard for beginners. C−− obtains alias freedom more crudely but far more simply: every collection is a value, copied on assignment and argument passing.

Subsets and verification. MISRA C [11] restricts C through coding rules enforced by external checkers, and SPARK [12] restricts Ada so that programs can be formally verified. C−− shares their subsetting philosophy but enforces the restrictions in the grammar and type system of a standalone language, so that a violating program cannot be written at all.

Error messages. Research on novice programmers shows that terse compiler messages are a major obstacle [13], and languages such as Elm popularized messages written for humans [14]. C−− adopts this style for every one of its diagnostics.

III. LANGUAGE DESIGN

A. Principles

Three rules govern every design decision. *(i) Remove rather than guard:* a feature whose misuse causes a vulnerability class is omitted unless it can be given fully defined semantics. *(ii) No undefined behaviour:* every program either has a precisely specified result or stops with a precisely specified, positioned error. *(iii) Explicitness over convenience:* there are no implicit narrowing conversions, no truthiness and no operators whose meaning depends on context.

B. Types and values

C−− has four primitive types and four generic collection types (Table I). Integers are 64-bit two's-complement values whose every operation is range-checked; division and remainder truncate toward zero, so the identity $(a/b)\cdot b + a \bmod b = a$ holds for all non-zero divisors. Floats follow IEEE 754 binary64 [15], but infinities and NaN can never be produced: an overflow or a division by zero is a runtime error instead. The only implicit conversion is the widening of an `int` operand to `float` in mixed arithmetic and comparison; assignment never converts.

```
module main;

fn mean(xs: list<float>) -> float {
    mut total = 0.0;
    for (mut i = 0; i < xs.len(); i = i + 1) {
        total += xs[i];
    }
    return total / xs.len();
}

fn main() {
    mut samples = list<float>();
    samples.push(2.5);
    samples.push(4.0);
    let snapshot = samples;    // deep copy, not an alias
    samples.push(9.5);
    print(mean(samples));      // 5.333333333333333
    print(snapshot.len());     // 2
}
```

Listing 1. Typed functions, a growable list and deep-copy value semantics.

C. Bindings and mutability

A variable is introduced with `let` (immutable) or `mut` (mutable) and *must* be initialized; the grammar has no declaration without an initializer, so an uninitialized read cannot be written. Types are inferred from the initializer. Immutability is deep: on a `let` collection neither element assignment (E029) nor mutating methods (E028) are allowed. Function parameters are immutable.

D. Statements and control flow

Assignment is a statement, not an expression, so `if (x = 1)` and `a = b = 0` are syntax errors. The operators `++`, `--`, single `&` and single `|` are rejected by the lexer with a hint to use `+= 1`, `-= 1`, `&&` and `||`. Braces are mandatory for every body, which removes the dangling-`else` problem. Conditions must have type `bool`. A `switch` matches `int` or `string` literals, has no fall-through and rejects duplicate cases at compile time.

E. Functions

Functions have typed parameters and an optional return type. The analyzer collects all signatures before checking bodies, so definition order is free and mutual recursion works. A conservative *definitely-returns* rule rejects any typed function with a path that falls off the end (E013): a block returns if any statement in it returns, an `if` only if it has an `else` and all branches return, and loops never count. There are no globals; a function sees only its parameters. Recursion depth is bounded at 1000 calls (R007) on every platform.

F. Collections and value semantics

Collections never alias. Assigning, passing or returning a collection copies it, so a callee can never modify its caller's data and two names never observe each other's updates (Listing 1). This removes the aliasing that ownership systems must otherwise track, at the cost of copying, which we accept in exchange for a much simpler mental model.

Table IV summarizes how each language rule maps to the weakness class it removes.

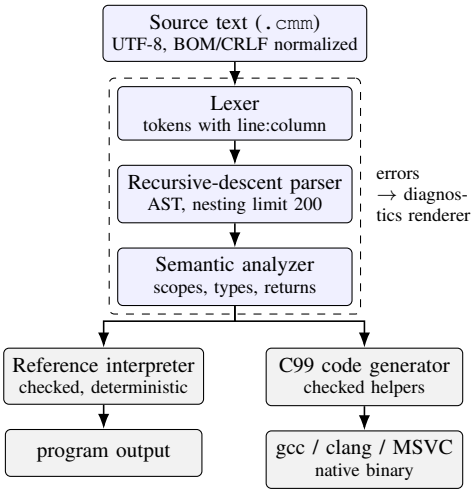


Fig. 1. Compilation pipeline. Each stage reports positioned errors to a shared diagnostics renderer.

IV. Implementation

The reference implementation is about 6 300 lines of Python using only the standard library, so it runs unchanged on Windows, macOS and Linux. Fig. 1 shows its structure.

A. Front end

The source is decoded as strict UTF-8; a byte-order mark is removed and line endings are normalized before lexing, and columns count Unicode code points, so positions are identical on every platform. The hand-written lexer uses longest-match tokenization and stops at the first lexical error. The parser is a stratified recursive-descent parser [16] with one procedure per precedence level (logical or, logical and, equality, comparison, additive, multiplicative, prefix and postfix), giving C's usual precedence and left associativity. A nesting limit of 200 levels (S004) bounds recursion on adversarial input.

B. Semantic analysis

The analyzer performs two passes: it first records every function signature, then checks each body with a stack of lexical scopes. It resolves names, infers and checks types, enforces `let`/`mut` rules, validates calls and method calls against a table of built-in collection methods, rejects duplicate `switch` cases and applies the definitely-returns rule. All semantic errors in a program are collected and reported in source order, rather than only the first one.

C. Reference interpreter

The normative semantics are given by a tree-walking interpreter. Every integer operation is checked against $[-2^{63}, 2^{63}-1]$ and every float result against overflow; division by zero traps for both types. Collections are copied on assignment and argument passing. Each call creates a fresh environment holding only its parameters. To make the 1000-call depth limit behave identically on all platforms, the interpreter runs in a thread with a 256 MB stack. An optional step budget (`--max-steps`) bounds runaway loops, and the web server uses it for untrusted code.

TABLE II
Size of the reference implementation

Component (modules)	Lines
Lexer and token definitions (`lexer`, `tokens`)	585
Parser and AST (`parser`, `ast_nodes`)	1 343
Semantic analyzer and type rules (`analyzer`, `types`)	778
Interpreter and runtime (`interpreter`, `runtime_*`, `builtins`)	1 117
C99 code generator (`codegen_c`)	859
Diagnostics (`errors`, `explanations`, `suggest`, `diagnostic_render`)	867
CLI and printers (`cli`, `astprint`, `formatting`, others)	755
Total	**6 304**

D. ISO C99 back end

The transpiler lowers the v0.1 subset plus user-defined functions to portable C99 that uses only standard headers. Arithmetic is emitted as calls to small `static` helpers that perform the same overflow and division checks as the interpreter, and a custom formatter reproduces the interpreter's shortest round-trip float printing, so a transpiled program prints exactly what the interpreter prints. Collections, indexing and `switch` are not yet supported by this back end; such programs are rejected with error T001 rather than being compiled incorrectly.

E. Tooling

A command-line tool provides `run`, `check`, `tokens`, `ast` and `transpile` commands with distinct exit codes for static errors, runtime traps and I/O failures. A zero-dependency web studio exposes the same pipeline through a JSON API and a browser editor with syntax highlighting, live diagnostics as the user types, and inspectors for tokens, the AST and the generated C99.

V. Diagnostics

Because C−− targets learners as well as careful programmers, every error is designed to be read by a novice [13]. Each of the 60 diagnostic codes (Table III) has a positioned headline, a source excerpt with a caret under the offending token, a plain-language explanation under *what went wrong*, and a concrete repair under *how to fix*, often with corrected code. For an undeclared name, the analyzer proposes the closest declared identifier by Damerau–Levenshtein distance [17], [18], accepting at most one edit for names shorter than five characters and two otherwise (Listing 2). A `--no-help` flag restores a terse three-line format for tools.

VI. Evaluation

A. Safety guarantees

Table IV maps eleven weakness classes to the C−− rule that removes them and to the stage at which a violating program is stopped. Classes in the upper part cannot be expressed at all; those in the lower part are expressible but become deterministic, positioned runtime traps instead of undefined behaviour. Each code in the table is exercised by a program in the test corpus.

TABLE III
DIAGNOSTIC CODES BY PHASE

Prefix	Phase	Codes	Examples
L	Lexical	12	++ not supported, unterminated string
S	Syntax	3	expected token, reserved word, nesting
E	Semantic	30	undeclared name, `let` reassigned, missing return
R	Runtime	14	overflow, division by zero, index range
T	Transpile	1	feature not supported by the C back end
	Total	**60**	

```
typo.cmm:5:19: semantic error: undeclared variable
       'maximum_scor'
  |
5 |     mut current = maximum_scor + 10;
  |                   ^^^^^^^^^^^^
what went wrong:
  'maximum_scor' has not been declared in this scope.
how to fix:
  Declare 'maximum_scor' before using it, with 'let
       maximum_scor = ...;' or
  'mut maximum_scor = ...;'.
      let maximum_scor = 0;
  did you mean 'maximum_score'?
```

Listing 2. Diagnostic for a misspelled identifier, as printed by `cmm run`.

B. Test suite

The implementation is covered by 445 automated tests: unit tests for each phase, golden-output tests for all example programs, a corpus of 61 programs that each trigger one diagnostic and compare the full message character for character, tests that run every example through both back ends and require identical output, and tests for the web server. On our machine 444 tests pass. The single failure is a portability defect in the C back end, not in the language: for a literal equal to $2^{63}-1$ the generated code prints an `LL` constant with the `PRId64` format, which GCC rejects under `-Werror` on platforms where `int64_t` is `long`. The fix is to cast the constant to `int64_t`.

C. Performance

Table V reports two experiments on four benchmarks covering recursion, integer loops with division, data-dependent control flow and floating point. The first compares the interpreter with the same program transpiled to C99 and compiled with `gcc -O2`; all outputs matched between the two back ends. The second measures the cost of C−−'s checked arithmetic by comparing the transpiled C99 with hand-written, unchecked C at larger problem sizes. Times are the best of several runs on Intel Xeon Processor @ 2.10GHz, with GCC 13.3.0 and Python 3.13.16. Native times include process start-up, which dominates at the small sizes of Experiment 1, so its speedups are lower bounds.

The interpreter is meant to define semantics and to support teaching, not to be fast: the transpiled C99 runs the same programs 225–878 $\times$ faster. The checked C99 is 29–217% slower than unchecked hand-written C, and this overhead is the price of the guarantees. Every arithmetic operation becomes a call to a small checking helper that adds a comparison and a branch, and the checks restrict optimization. The cost is lowest where work is dominated by control flow (Collatz) and highest where tight arithmetic would otherwise be simplified aggressively: recursive Fibonacci, and the floating-point series, whose every division is checked for a zero divisor and every result for overflow. Reducing this cost, for example by proving ranges statically and omitting redundant checks, is future work.

D. Application: modelling SIMD predication

C−−'s value semantics make it a convenient modelling language. In a separate case study we modelled AVX-512 vector registers as `array<float>` values and mask registers as `array<bool>` values to study how predication removes branches from a conditional loop. The bounds checks proved useful there: removing the tail mask from a modelled load made the program stop with R006 instead of silently reading past the array, which is exactly the fault that hardware masked loads exist to prevent [19].

VII. Limitations and Future Work

C−− deliberately trades expressiveness for safety. It has no structs, enumerations, maps, nested collections, first-class functions or string concatenation, and copying collections on every assignment is expensive for large data; copy-on-write would preserve the semantics at lower cost. The C99 back end does not yet support collections, indexing or `switch`. The interpreter is slow by design. Planned work includes user-defined record types, copy-on-write collections, extending the C back end to the full language, and a formal semantics against which the interpreter can be verified.

VIII. Conclusion

C−− shows that removing features, rather than adding protective machinery, yields a small imperative language in which the most common memory-safety and undefined-behaviour defects either cannot be written or stop deterministically with a clear, positioned explanation. The language keeps C's explicit, readable style; its implementation is compact, portable and dependency-free; and its diagnostics are written for the people who most often meet them. The source code, documentation and web studio are openly available [20].

Acknowledgment

The authors thank Prof. Hariharan R L, Center for AI Research, School of Computer Science and Engineering (SCOPE), for his guidance and supervision of this project.

References

[1] M. Miller, "Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape," Presented at BlueHat IL, Tel Aviv, Israel, 2019.

[2] The Chromium Projects, "Memory safety," https://www.chromium.org/Home/chromium-security/memory-safety/, accessed: Oct. 2026.

TABLE IV
WEAKNESS CLASSES AND HOW C−− REMOVES OR TRAPS THEM

Weakness (CWE)	**C−− rule**	**Enforced by**
NULL dereference (CWE-476)	No null value and no pointer types exist	grammar and type system
Buffer overflow via pointers (CWE-119, CWE-787)	No address-of, dereference or pointer arithmetic	grammar
Use of uninitialized variable (CWE-457)	Every declaration requires an initializer	parser (S001)
Use after free, aliasing bugs (CWE-416)	No references; collections are copied on assignment and call	semantics
Assignment in condition, ++/−− misuse	Assignment is a statement; `++`, `--`, `&`, `|` rejected	parser, lexer (L009–L012)
Implicit conversion, truthiness (CWE-704)	Conditions must be `bool`; assignment never converts	analyzer (E004, E007)
Accidental mutation	`let` bindings and parameters are deeply immutable	analyzer (E003, E028, E029)
Switch fall-through (CWE-484)	No fall-through; duplicate cases rejected	semantics, analyzer (E023)
Out-of-bounds index (CWE-125, CWE-787)	Every index is checked	runtime trap (R006)
Integer overflow (CWE-190)	All `int` operations checked in 64 bits	runtime trap (R001)
Division by zero (CWE-369)	Checked for `/` and `%` on both types	runtime trap (R002, R003)
Uncontrolled recursion (CWE-674)	Depth limited to 1000 calls on every platform	runtime trap (R007)

TABLE V
PERFORMANCE: INTERPRETER VS. TRANSPILED C99, AND CHECKED C99 VS. UNCHECKED HAND-WRITTEN C

	Experiment 1: execution back ends				**Experiment 2: cost of checks**			
Benchmark	N	**Interp. (s)**	**C99 (ms)**	**Speedup**	N	**Checked C99 (s)**	**Plain C (s)**	**Overhead**
Recursive Fibonacci	22	0.37	1.6	225×	35	0.046	0.018	+152%
Prime counting (trial division)	20 000	1.89	3.4	554×	2 000 000	0.989	0.566	+75%
Collatz step totals	3 000	1.50	1.7	878×	1 000 000	0.267	0.206	+29%
Leibniz series for π (float)	200 000	1.48	2.1	712×	200 000 000	0.675	0.213	+217%

[3] Cybersecurity and Infrastructure Security Agency (CISA) et al., "The case for memory safe roadmaps," CISA, NSA, FBI, and international partners, Tech. Rep., Dec. 2023.

[4] T. Jim, J. G. Morrisett, D. Grossman, M. W. Hicks, J. Cheney, and Y. Wang, "Cyclone: A safe dialect of C," in *Proc. USENIX Annual Technical Conference*, 2002, pp. 275–288.

[5] G. C. Necula, S. McPeak, and W. Weimer, "CCured: Type-safe retrofitting of legacy code," in *Proc. 29th ACM SIGPLAN-SIGACT Symp. Principles of Programming Languages (POPL)*, 2002, pp. 128–139.

[6] A. S. Elliott, A. Ruef, M. Hicks, and D. Tarditi, "Checked C: Making C safe by extension," in *Proc. IEEE Cybersecurity Development (SecDev)*, 2018, pp. 53–60.

[7] N. D. Matsakis and F. S. K. II, "The Rust language," *ACM SIGAda Ada Letters*, vol. 34, no. 3, pp. 103–104, 2014.

[8] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *Proc. USENIX Annual Technical Conference*, 2012, pp. 309–318.

[9] ISO/IEC, "Programming languages — C," ISO/IEC 9899:1999, 1999.

[10] S. P. Jones, N. Ramsey, and F. Reig, "C--: A portable assembly language that supports garbage collection," in *Proc. Int. Conf. Principles and Practice of Declarative Programming (PPDP)*, ser. LNCS, vol. 1702, 1999, pp. 1–28.

[11] MISRA, *MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems*. MIRA Ltd., 2013.

[12] J. Barnes, *High Integrity Software: The SPARK Approach to Safety and Security*. Addison-Wesley, 2003.

[13] B. A. Becker, P. Denny, R. Pettit, D. Bouchard, D. J. Bouvier, B. Harrington, A. Kamil, A. Karkare, C. McDonald, P.-M. Osera, J. L. Pearce, and J. Prather, "Compiler error messages considered unhelpful: The landscape of text-based programming error message research," in *Proc. Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR)*, 2019, pp. 177–210.

[14] E. Czaplicki, "Compiler errors for humans," https://elm-lang.org/news/compiler-errors-for-humans, 2015.

[15] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std. 754-2019, 2019.

[16] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.

[17] F. J. Damerau, "A technique for computer detection and correction of spelling errors," *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964.

[18] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[19] J. R. Allen, K. Kennedy, C. Porterfield, and J. Warren, "Conversion of control dependence to data dependence," in *Proc. 10th ACM SIGACT-SIGPLAN Symp. Principles of Programming Languages (POPL)*, 1983, pp. 177–189.

[20] R. Mexson, M. K. R. Panchal, and G. Gupta, "C-- (C Minus Minus): Language, Compiler, and Web Studio," https://github.com/ronitmexson2924/C--, 2026.